

Data Book of truth

(no semantic web / XML lecture, no drawing star / snowflake schema, no self efficacy, no OLAP cubes)

Mongo DB:

A document DBMS which we can use python to query

Queries:

Basic query is in the form:

```
query = {"field": "value"}
```

E.g.:

```
Query = {"Studentno": 24}
```

This returns every student with a student number of 24 (of which There will only be one)

Queries must be ran on a collection of data using:

```
recordList = col.find(query)
```

This runs the query, each returned record is stored in recordList

The records can be printed by simply iterating through the list:

```
For record in recordList:  
    print(record)
```

More complex queries that use operators are in the form:

```
complexQuery = {"$and":[{"Age": 24}, {"Country":  
"Britain"}]}
```

This returns every 24 year old in Britain

Or

```
veryComplexQuery = {"$or": [{"StudentName": "Bob"}, {"Grade": {"$lte": 30}}]}
```

This query returns Every student named 'Bob' Or with grades lower than or equal to 30%

Regex can also be used to return records where a field contains certain words:

```
regexQuery = {"$not": [{"Name": {"$regex": "LIGHTS"}}]}
```

This query returns all products which aren't lights

Operators:

Logical operators can be used to return relevant records, they are:

- `$or` - returning records that fulfil either condition
- `$and` - returning records that fulfil all conditions
- `$lt(e)` - returning elements less than (or equal to) a specified value
- `$gt(e)` - returning elements greater than (or equal to) a specified value
- `$not` - inverts the expression so returns records that do not fulfil condition

Usage for operators can be seen above

Big Data:

Big Data = Huge amounts of data that is too complex or too large to be processed or managed using conventional techniques, usually including unstandardised data in many forms (records of people / records of sales with different fields in the same table)

Big Data has 4 attributes:

- **Volume** - The amount of data present
- **Velocity** - The rate at which new data is recorded
- **Variety** - How many different forms the data is in: structured, semi-structured or unstructured data
- **Veracity** - Amount of inconsistency in data

Structured Data = data in a predetermined, well structured format (i.e. records in relational DB)

Semi-Structured Data = Data is not in a predetermined, well structured format but has some level of organisation (i.e. XML)

Unstructured data = Data that is not in any kind of format that is very hard to process (i.e. text files on a hard drive)

Examples of big data include:

- The new york stock exchange storing trade data
- Uploads to social media

Issues with Big Data:

- **Data is not knowledge** - Data must be processed to obtain insights from it and more data does not always mean we have more information, data must be relevant, interpretable and contextual to be useful
- **Data is difficult to process** - Big data is difficult and cost / work intensive to process due to its sheer size, complexity and containing data in a variety of forms
- **Data is often collected opportunistically** - Data is obtained wherever it can be, this means that a lot of data in big data will be useless

- **Legal / Ethical Considerations** - There is a lot of data present so there are many legal / ethical considerations present when handling big data

Benefits of Big Data:

- **Gaining insight** - trends can be found in this data which organisations can use to make better decisions (i.e. increasing prices when product more in demand)
- **Scientific discovery** - trends in data can illuminate scientific processes (i.e. climate change discovered due to monitoring of temperature and CO2 in air, both big data)
- **Data is cheap** - Collecting data often has a small cost and can be done passively

Types of database:

Relational Database:

Relational Database = A database composed of tables, each containing data in the form of records

Each table contains a primary key which uniquely identifies an entity and prevents duplication of entities.

Tables can also be related to each other via the use of foreign keys which relate entities in one table to the entities in another, enforcing referential integrity.

When properly set up, relational databases adhere to normal form rules which prevent anomalies in the data and data duplication

Advantages -

- Data is always consistent and integrity is high
- There is no duplication of data

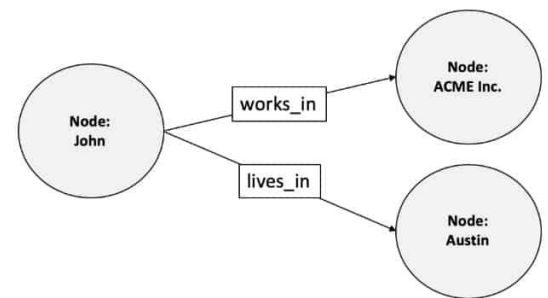
Disadvantages -

- Unsuitable to large volumes of data as if a relational database is very large, performance issues will be present as there will be many related tables, each of which will need to be joined when querying
- Unsuitable to a large variety of data as many columns will not be used by the majority of records, thus leading to lots of wasted space

Thus a relational database is unsuitable for use with big data

Graph Database:

Graph Database = A database represented by a graph where nodes represent individual entities and edges represent the relationships between the entities



Advantages -

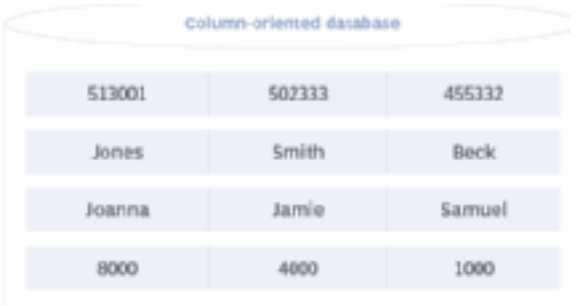
- Flexible schema, nodes do not have to store the same data so each node can represent a new type of entity
- We can gain more insight as they allow us to easily identify and focus on relationships in data that may be missed in other databases
- Querying is efficient as joins are not required, nodes are just traversed

Disadvantages -

- Not suited to large volumes of data due to limited scalability, as complexity grows exponentially with each new node, so speed of traversal decreases

Column Stores:

Column Store = A database where data is stored in tables akin to a relational database. Unlike a relational database, each entity is represented by a column



A diagram of a column-oriented database represented as a cylinder. Inside the cylinder is a table with three columns and four rows. The columns are labeled with IDs: 513001, 502333, and 455132. The rows contain names: Jones, Smith, Beck, Joanna, Jamie, Samuel, and numerical values: 8000, 4000, 1000.

513001	502333	455132
Jones	Smith	Beck
Joanna	Jamie	Samuel
8000	4000	1000

Advantages -

- Suited to high data variance as No wasted storage as not all rows must have the same n.o. columns as each other
- Query performance of a single column is high as columns do not have null values that need to be read and only 1 column from storage is needed
- Scalable when adding entities as new column can simply be added

Disadvantages -

- Query performance of searching rows is much slower than in a relational DB as each value for specified row must be extracted from each column before the search
- Not very scalable when adding attributes as data would have to be added at the end of each column, instead of all at once in a single operation in a relational DB

Key Value Store:

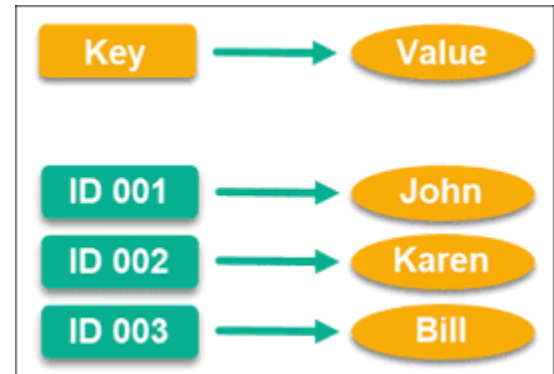
Hashing:

Hashing = used in Key value stores, performing a mathematical function on a key string to irreversibly transform it into a unique integer which can be used to determine the position of the key element in a hash table. This operation has a $O(1)$ complexity so does not take longer if more elements are present

Collisions can occur between hashed values when using a poor hash function however, where different strings are transformed into the same value, leading to errors where different values are assigned the same position in a hash table

Definition:

Key Value Store = A database composed of an array of simple keys, which each, when hashed, identify the position of a complex value / record in a hash table (e.g. A student number mapping to the full info about the student)



Keys must be unique to avoid key collisions where 2 records will be assigned the same position in the hash table, keys must also be sufficiently short as to not take too long to process

The values the keys map to can be anything, so an entire entity could be represented as a value, or only a single value could be stored, this is because each key value pair is independent of all of the others

Key value stores also do lack referential integrity due to the lack of a predetermined schema and each key value pair being treated as a single entity, so no foreign keys to enforce referential integrity

Advantages -

- High query performance as hashing has a complexity of $O(1)$ and referential integrity is not needed, as such a query can be completed by simply hashing the key and looking up the corresponding position
- Well suited to high volumes of data as hashing has a $O(1)$ complexity so there are no performance issues when querying large amounts of data
- Well suited to high data variance as each value in a hash table can store different data types or even entire elements

Disadvantages -

- Not well suited at high data veracity as no referential integrity, as foreign keys are not present since each key value pair is independent so nothing can enforce referential integrity, as such data may

contradict itself, being difficult to process, this may pose challenges if data is already inconsistent

- Can only perform basic queries by looking up the value corresponding to the key, cannot do more complex queries like SQL, can only PUT (add data), GET (output data), DELETE (delete data)
- Collisions can occur which lead to loss of data

Document Databases:

Document Database = Similar to a key value store where a simple key string identifies a self contained document which itself can contain many elements / pieces of data

When querying key value stores, the entire value corresponding to the key is outputted, whereas in document databases, documents themselves can be queried to output individual pieces of data of the document

Essentially a key value store which stores tables

Each document is also independent, and can be used to store different elements

MongoDB is an example of Document Databases

GERMANY

Advantages -

- Well suited to high data variance as each document is independent so can contain different fields
- High query performance due to key value approach of hashing keys to find position of document and no need to join tables

Disadvantages -

- Still only basic queries can be used, more complex than a key value store as documents themselves can be queried but not as complex as SQL

- No Normalisation or referential integrity in documents as they are each independent of each other, data can be duplicated between documents and inconsistencies can appear as changes to data are not cascaded throughout all documents, causing data to contradict itself

Transactions:

Transaction (OLTP) = exchange of data between 2 databases or between tables in a database, while maintaining referential integrity of data, it does this by adhering to ACID properties

When a transaction is completed, the changes are committed to the DB, when a problem arises, the DB is rolled back

Acid -

- **Atomicity** - the transaction must be completed fully and all values must be updated in each table, or a database is rolled back to a previous version
- **Consistency** - The state of the database must remain consistent, i.e. the total amount of money in a bank account DB must remain constant
- **Isolation** - Each transaction must occur separate from each other one, so one transaction in a record must be completed before another one begins, this is enforced using record locking
- **Durability** - Databases are saved in secondary storage so they can be rolled back if hardware fails

Problems with transactions:

Lost Update Problem = When multiple transactions are not completed in isolation, a value can be updated concurrently, leading to the initial transaction being completed but a value being overwritten by the new transaction, thus ignoring the first

E.g. when 2 transactions remove money from a balance and add it to another account unique to the transaction. If the balance is £5000, and the first transaction removes £100 from this balance and adds it to the balance of another account, the balance would be £4900, and £100 would be added to the balance of the other account, if another transaction occurs concurrently however, that removes £5 from this balance and adds it to an account, the previous transaction's effect on the account's balance will be overridden, and the value will be set to £4995, with the £100 being added to the other account's balance, this change will then be committed so £95 would be created from thin air

Dirty Read Problem = If concurrent transactions occur, one transaction may update a stored value, then it may be read by the next transaction, the initial transaction may encounter an error and roll back to the previous version before the change, as such the value stored by the DB and the value read by the 2nd transaction are not equal

E.g. if a transaction adds £300 to a bank account of balance £100, the balance value is updated to £400, then a 2nd transaction removes all the balance from an account, adding it to another account, the balance is then read as £400 and £400 is added to the account, then an error occurs in the initial transaction so the values are rolled back, so the £300 is added back to the previous account and the balance is set to £100, then it is set to 0 by the 2nd transaction, ergo £300 would be created from thin air

Inconsistent analysis problem = If concurrent transactions occur, one transaction may read many values from a DB while other transactions are modifying the values in it, as a result, a mixture of outdated and newer values are read, leading to an inaccurate result of the calculation

E.g. when calculating the average bank balance of many accounts while transactions occur to add / remove money to the accounts, a mixture of old values before many transactions occurred will be read and newer values after the transactions occurred will also be read, leading to an inaccurate average balance

How to solve problems:

Making all transactions occur serially - making all transactions occur serially will prevent any of these problems occurring but lead to massive performance issues when multiple users use the DB as each transaction would only occur after the previous one is fully completed so users will have to wait for the previous transaction to be completed before they can perform their own

Record locking:

When a transaction which writes to a record is occurring, the transaction can lock a record to prevent access to it until the transaction is complete to prevent data being incorrectly read or updated while still allowing for concurrent transactions, increasing performance

Transactions which read data do not lock records as this is not needed and would hinder performance, only modifying transactions lock records

Two Phase Locking = a more advanced form of record locking used when multiple data items are being updated, made up of two phases:

- Growing phase - locks are acquired and applied to data items being modified, so transaction has access to whole data being modified
- Shrinking phase - When transaction is complete, locks are released, only begins to release locks after all locks are acquired and data is modified

However, this approach has issues:

Deadlock = when multiple transactions occur and lock records simultaneously, they will each wait for the other transaction to be completed and the lock to be released but this will never happen, as such preventing access to certain rows

There are however many ways to overcome deadlock -

- **Timeouts** - transactions are rolled back if they take too long to complete (because of deadlock), this would release the locks they held on records
- **Deadlock detection** - the transaction which would be easiest to stop is rolled back, lifting its lock and allowing the other transaction to access the data
- **Deadlock prevention** - prevent deadlocks occurring altogether by analysing which transactions access which records and the DBMS scheduling transactions

Distributed Databases:

Distributed Databases = Databases where the data being stored is in different physical locations, with the DBMS making this invisible to end users

This allows for the use of more storage so more data can be stored but introduces potential performance issues as now network speed must be taken into account

The CAP theorem applies to distributed databases

CAP Theorem:

Consistency - The database is in a consistent state over the network, this can be enforced by rolling back every transaction when errors are observed, this ensures that all copies of the DB on different machines are identical

Availability - Every remote query must get a response from the database and be given data, although running the risk that this is not the most recent write, the database can still be accessed, even when network errors occur

Partition tolerance - The database remains active despite network failures or failures of individual nodes

The CAP theorem states that only 2 of these 3 attributes can be applied to a distributed database at any one time

Since Partition tolerance is required due to the nature of distributed databases, designers must choose between a consistent and constantly available database (i.e. the database has the choice of responding to queries despite network errors)

Essentially a DB can prioritise availability over consistency and respond to every query despite the network issues causing the database to output incorrect values, ergo if a change is made to one copy of the database, the other copy will return the old, incorrect value as the network prevents it from being updated

If a DB prioritises consistency over availability, the database will not respond to any queries to prevent a value being changed in one DB and not in another, keeping data consistent across the network

Transaction Recovery:

Logging = recording all the changes made to a DB in a log file, including before and after values, this allows the database to be rolled back to a previous version in case of a transaction error or hardware failure

Checkpoints are also made periodically, committed transactions are suspended until a checkpoint is complete, these transactions are only written to the disk when a checkpoint is complete

in the checkpoint prior to failure, incomplete transactions must be rolled back and committed transactions must be reapplied to the DB, transactions committed in the previous checkpoint do not need to be reapplied as they have been written to the disk

The log file is required to reapply changes as committed transactions will be included in the log and incomplete transactions will not be, as such it must be stored in a separate storage medium to the DB so it is not also lost when hardware fails

Archiving = copying log files in offline storage every so often so the DB can be reloaded from old logs, before the changes in the active log file are reapplied

Methods of increasing DB performance:

Indexing:

Indexing = storing a small, ordered subset of a table's columns (including frequently searched terms) which can be accessed while searching for a particular record, this means that not every record in the table must be scanned and sorted before the DB can be searched, greatly improving the performance of queries which access data

Fields in an index can point to records in the main table, so records can be accessed from the index when the corresponding value is found

The PK field is automatically indexed by DBMSs, DB admins can also index fields containing frequently searched terms

Since indexes are ordered, indexing fields which are commonly ordered in queries leads to high query performance when using these operations

There are 2 types of index:

Clustered index = An index where the indexed rows correspond to the table row where the desired record is held, leading to very fast queries, only one of these can appear per table as the records are ordered by this field and can only be ordered one way, this index is contained within the table

Non Clustered index = An index which is separate to the table itself contains pointers to the desired records in the table called row locators, leading to slower queries than the clustered index, despite still being faster than scanning the entire table, because these indexes are separate to the table, there can be an unlimited amount of them

Problems with indexing:

- Indexing should not be performed on smaller tables as it is slower to query the index and access the row from the row locator than simply to scan the table, indexing smaller table leads to lower query performance and wasting storage on these indexes
- Non Clustered Indexes must also be updated every time the DB is updated as they are separate to it, thus query performance can decrease if many fields in a frequently changing DB are indexed

Query Optimisation:

Query Optimisation = when the DBMS refines queries which the user has written to improve query performance and reduce resource consumption

e.g.

```
SELECT *
FROM Orders O
WHERE O.OrderDate > '2023-01-01'
AND O.CustomerID IN (
    SELECT CustomerID
    FROM Customers C
    WHERE C.City = 'New York'
);
```

In this example, the query returns the list of order records after 2024 where the customer is in New York

An optimiser in a DBMS would refine this to:

```
SELECT  
FROM Orders O  
INNER JOIN Customers C ON O.CustomerID = C.CustomerID  
WHERE O.OrderDate > '2023-01-01'  
AND C.City = 'New York';
```

This approach uses an inner join to directly combine the Customer and Order tables to avoid the subquery and reduce the n.o. table scans, improving the performance of this query

Data Warehouses

Data Warehouse = A large collection of historical, archived data from a variety of sources, kept separate from an organisation's operational DB (to prevent loss of operational DB and data warehouse if error occurs), which analysts can make informed decisions based on and is rarely added to, only when new data is no longer useful and ready to be archived.

Data warehouses provide a multidimensional view of the data

Data is organised into dimension tables using star / snowflake schema

Data warehouses are typically denormalized to improve query performance (due to the vast amount of data present, normalising it would make queries incredibly complex) and since data is presented in different dimensions, some data duplication is required

Data warehouses also do not replace / alter the original data from the sources, they will simply store copies of the data that is updated (refreshed) periodically.

Generally speaking, only larger companies use data warehouses due to their cost to create / maintain, the volumes of data needed to be stored (as smaller companies simply will not have enough data

Features:

A data warehouse is...

- **Subject Oriented** - Data is organised around specific subjects, making modelling / analysing data easier. Operational DBs use a process oriented approach and focus on the day to day operations of an organisation
- **Integrated** - Data in a data warehouse is from a variety of sources (relational DBs, flat files, etc, all with different fields). Data is cleaned and standardised when it is added to the warehouse, ensuring that all data, though it comes from a variety of sources, are in a consistent format suitable for analysis, or alternatively, data is not cleaned and each query is adapted for each data source, however, this leads to low query performance but is easy to add new data
- **Time variant** - Data in a data warehouse is time stamped to capture their state at different times (so the time when they are added is recorded), this allows for analysis of trends over different timeframes
- **Non Volatile** - Data in a data warehouse is not changed once it is added, this allows for a preserved historical record of data

Types of applications which run on a data warehouse:

There are 3 types of applications which use data warehouses:

- **Information Processing** - processing the data stored in a data warehouse by performing queries to retrieve data from a warehouse, then performing basic statistical analysis of the data (averages, sums, etc), the results of queries / basic statistical analysis are also presented as reports in a variety of formats
- **Analytical Processing (OLAP)** - supports the analytical processing of stored data by using OLAP operations

- **Data Mining** - Extracting hidden trends in the vast amounts of data stored in a data warehouse, using sophisticated algorithms to extract information that would be missed by simpler methods

Data Warehouse (OLAP)	Operational Database (OLTP)
Historical processing of information	Day-to-day processing
Used by knowledge workers – managers and analysts	Used by clerks, DBAs or DB professionals
Analyse the business	Run the business
Focuses on information OUT	Focuses on information IN
Based on Star Schema, Snowflake Schema, Fact Constellation Schema	Based on Entity Relationship Model (commonly)
Historical Data	Current Data
Summarised and consolidated data	Primitive and highly detailed data
Multidimensional view of data	Detailed and flat relational view of data
Number of users in 100s	Number of users in 1000s
Number of records accessed in millions	Number of records accessed is in 10s
Highly Flexible	High Performance
Size is 100GB-100TB	Size is 100MB to 100GB

Enterprise Data Warehouse:

Enterprise Data Warehouse (EDW) = A data warehouse which acts as a centralised repository, storing all the historical data of an organisation, as such, a vast amount of historical data is stored, spanning a large amount of time, it must also cater to the needs of everyone in an organisation who will use it

An EDW may have sub data warehouses or data marts to cater to the specific needs of individual staff in an organisation, they also often feature an operational data store

Most data warehouses are EDWs

Data Mart = A subsection of a data warehouse housing more specific, focused information relating to a single dimension (e.g. a data warehouse

may store info about all customers, a data mart may store the info about customers in the UK)

Operational Data Store = a DB which stores the most recent data for a real time snapshot of the state of operations (not an operational DB, an operational DB is process oriented)

ETL:

ETL = the process of adding new data to a data warehouse (Extract, Transform, Load)

The process of ETL is outlined here:

- **Data Extraction** - gathers a dataset of necessary data from different, diverse sources
- **Data Cleaning** - Identifies and corrects inaccurate / incomplete / irrelevant data in a dataset before the dataset is loaded into a data warehouse, this improves the quality of data in a data warehouse
- **Data Transformation** - converts raw data from the dataset into the warehouse format which is suitable for processing / analysis, uses a separate staging DB to apply these changes
- **Data Loading** - moving the transformed dataset into the data warehouse. While data is being moved, the integrity of the data is checked. Data is also indexed once it is moved
- **Refreshing** - updating data in a data warehouse with up to date information from sources to keep a data warehouse accurate

There is an alternative to ETL however called ELT

ELT = an alternative method of populating a data warehouse, extracts data from a variety of diverse sources and loads them directly into a data warehouse before transforming the data into a more suitable format

Designing Data Warehouse:

Top Down approach (Inmon):

Top Down approach = a method of designing data warehouses, starts with a central, highly normalised data warehouse containing diverse data from all operational DBs, this centralised data warehouse is normalised to 3NF / 4NF and uses an ER model. Data marts are then derived from this central warehouse

Advantages:

- Data in the central warehouse has high data integrity and few inconsistencies
- Efficient storage and processing of data, as data is highly normalised so there is little data duplication

Disadvantages:

- Very complex (and expensive) to design and maintain as many tables are required due to the high volume and variance of data
- Very low query performance as many join statements needed to query the data warehouse due to number of tables needed

This approach provides a definite and consistent view of the data and as such is very strong but is very difficult to query and very complex / expensive, as such it is preferred by large companies with enough money to create, maintain and analyse one

Bottom Up approach (Kimball):

Bottom Up approach = an alternative method of designing data warehouses, uses dimensional modelling (star / snowflake schema) to structure the data and to allow for efficient analysis of data. Data marts (each built for a specific dimension) are first built before being integrated into a central data warehouse. Data in this approach is denormalized (so

may be duplicated and inconsistencies may occur), to allow for efficient queries and analysis of data.

Advantages:

- Allows for efficient retrieval and analysis of data due to data being denormalized and dimensional modelling being used
- Very high query performance as queries are simple due to limited tables, no joins are needed
- Not as complex / cheaper than Inmon approach

Disadvantages:

- Data is not stored efficiently as data is not normalised, leading to a lot of data duplication
- Data has low integrity as it is not normalised

This approach provides efficient retrieval and analysis of data, it is also less complex and thus less expensive to create / maintain / analyse so is generally preferred by smaller companies with limited funds, however, this approach is not as strong as data is denormalized and inconsistent

Dimensional Data Model:

Dimensional Data Model = presenting data in the form of dimensions which each represent a different view of the data (e.g. a student dimension may present data about students, and a course dimension may present data about a course, data can be the same between the dimensions but they each represent different things)

Fact Table = central storage location data in a dimensional data model, stores quantifiable data (measurements) e.g. sales figures, profit etc., foreign keys link the data points in a fact table to the relevant dimension table

There are 3 types of fact table -

- **Transaction Fact Table** = records measurements related to individual events (e.g. sales of a product)
- **Snapshot Fact Table** = records timestamps which are explained by dimension tables
- **Accumulating Snapshot Table** = measurements in a fact table are incremented when new data appears

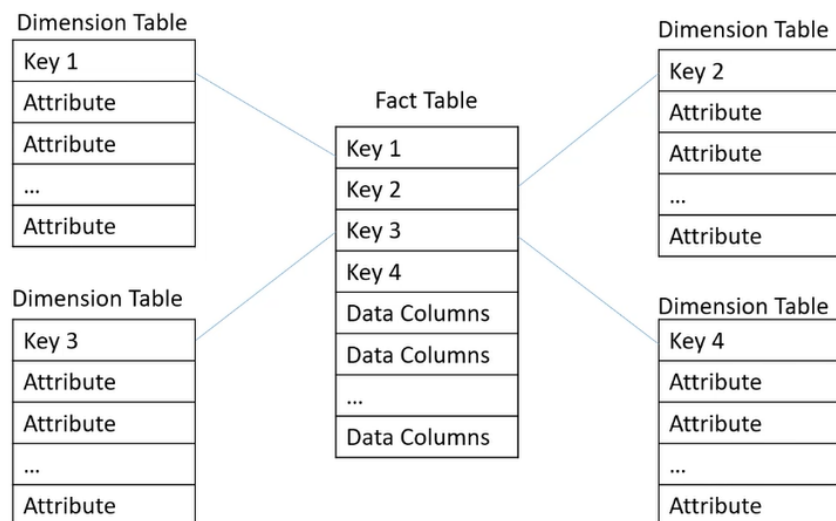
Dimension Table = storage location for the descriptive attributes of the measurements on a fact table, they provide context to these measurements based on the dimension the table represents, also often much smaller than a fact table, e.g. there may be a time dimension table which relates to the timestamps in a fact table and provides context to each of these timestamps

Dimension tables store a primary key (usually an integer ID) which relates to the foreign key in the fact table, this relates the descriptive attributes in the dimension table to the measurements in the fact table

Star Schema:

This schema includes a single, central fact table including measurements and foreign keys which link it to many dimension tables, giving context to the measurements in the fact table

This approach is denormalised, there may be duplicate attributes in the dimension tables



Using this schema leads to high retrieval query performance, this is done by pulling information together into a central point, despite causing duplication, this makes read queries simpler. This also makes write queries more complex due to the repeated data, however, this is not an issue as data warehouses are not written to often

Advantages:

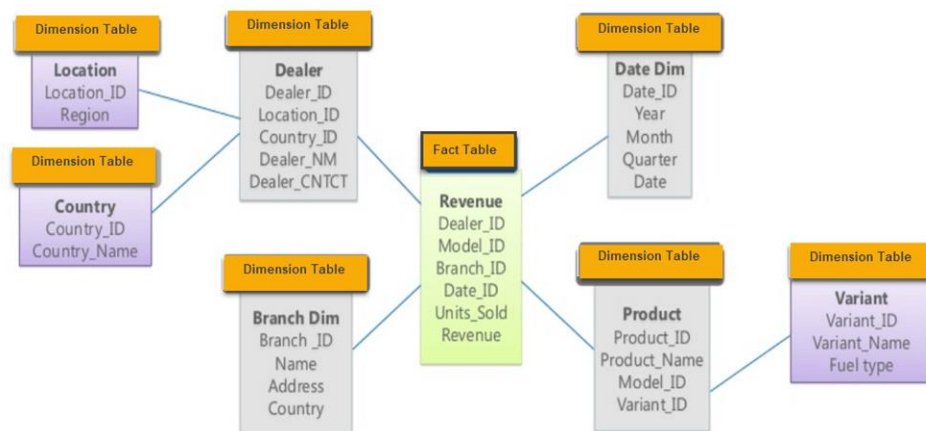
- Simple read queries and are simpler as few joins are used so data is retrieved and analysed more easily as all data connects through fact table
- OLAP cubes can also be built using star schema which allow for analysis of data
- Simpler to understand and design than snowflake schema

Disadvantages:

- Write queries are very slow due to the vast amount of repeated data, this is not much of an issue however as data warehouses are rarely written to
- Data integrity is also decreased due to denormalisation so inconsistencies may arise
- Complex queries that depend on relationships between data may be handled poorly as many to many relationships cannot be used and relationships between tables is strict

Snowflake Schema:

This schema also uses a single, central fact table connected to many dimension tables. However, in this approach dimension tables are normalised and broken down into a number of sub-dimension tables



Normalised data leads to high data integrity and fewer inconsistencies. however, read queries become more complex and thus slower as joins are required between the sub dimension tables

Advantages:

- Data is stored efficiently since data is normalised so data duplication is reduced, reducing storage required
- Data integrity / consistency is high since data is normalised
- Well suited to handling complex queries / analysis since relationships between data are shown

Disadvantages:

- Complex and thus expensive to design / understand due to increased relationships in data
- Read queries are slower than in a star schema as dimension tables are normalised so joins are required

Data Standards / Governance:

Data Standards = set of agreements that define how data is stored / formatted, using these effectively allows for efficient transfer of data (as data will be in the same format on multiple devices)

Data Governance:

Data Governance = How an organisation gathers, stores, processes and uses data, ensuring that data is secure and useful, in compliance with the data protection act, this covers the entire lifecycle of the data

Data Stewardship = The responsibility of data handlers to ensure that data is high quality and accurate, essentially managing the data. Usually a person or team within an organisation is responsible for this, one of the pillars of data governance

Data Protection Act (GDPR):

A law to preserve the privacy of users by imposing limits and rules on data which is stored

The GDPR states:

- Only the minimum amount of data about a subject should be recorded, e.g. data about customers in tescos shouldn't include customer ages as it is unnecessary
- Only relevant data about a subject is collected, again, unnecessary data should be avoided
- Stored data should be secure to prevent data leaks
- Data should be kept for the shortest time possible
- Relevant people (including users, government officials etc) can request access to their data and for data to be deleted

Anonymous data is not covered by the data protection act as it is anonymous and thus does not infringe the privacy of a person, however pseudonymous data does

Pseudonymous Data = Data which does not directly identify a person but can be used in accordance with other information to identify a person e.g. student number identifies students in a student DB but not on its own

Data value / cost:

Each piece of data has an associated value and cost

Data value = a measure of how useful and relevant data is, higher value data will provide an organisation more insights than low value data. e.g. historical record of an item's sales is high value as much can be inferred from it, but the average height of the people who buy it would have a low value as it provides no insights

Data Cost = The cost of gathering, storing and managing the data, data with a high cost may be difficult to obtain and incredibly large, e.g. a data warehouse of people's opinions on a topic over the previous decade, data with a low cost would be data which is small and easy to obtain, e.g. results of an anonymous online survey that few people complete

The value of the data should outweigh the cost of it, data with a high cost may still be useful as it has a very high value and vice versa. If the cost outweighs the value, organisations should be discarded.

To ensure that only relevant data is stored (and to reduce the cost of data storage / management) organisations should have data governance mechanisms, these mechanisms also allow for easy access to high quality data and archiving of low quality data

Data Quality:

Data Quality = A measure of how well data meets the needs it is required for. High quality data is accurate, contains all necessary information, is universally formatted, and adheres to relevant standards

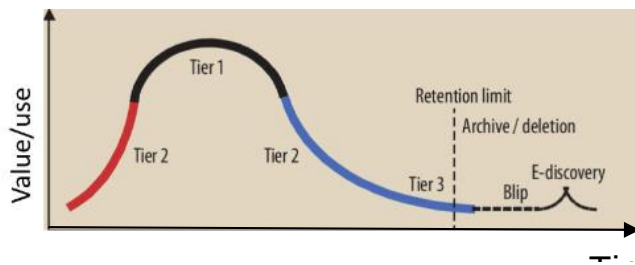
Raw data is usually of very low quality as it likely doesn't include context and is not formatted well

Raw data can be cleaned to format it in a correct way, correct errors in the data, or give data context. This is done by formatting data, recording metadata and merging different data sources to complete data

Cleaning the data makes it accurate, complete and consistent

Information Life Cycle:

A measure of the value of data at different times



Tier 1 - When data is cleaned and its value is maximised, should be easy to access

Tier 2 - When data has just been collected and has not yet been cleaned or is older and thus less relevant, these both reduce the value of the data, should still be accessible but not as easily

Tier 3 - Very old data that should be archived, it is very irrelevant and thus not useful but may still find some use after some time (e.g. data warehouse)

Data never has a value of 0

Information Retrieval and Search Engines:

The best example of information retrieval is a web search, it retrieves information based on a user's input

Most information on the web is unstructured

Issues with Information Retrieval:

- Only relevant results should be outputted, e.g. a search for musical bass shouldn't return information about the fish
- There should be measures in place to evaluate how well an IR system performs e.g. precision (How many returned results are relevant) and recall (How many relevant results were retrieved)
- User's intentions should be evaluated to return more relevant results (if a user searches for sea bass, all searches will return more information about fish)

Search Engine = an application which retrieves information from the world wide web and presents it to a user

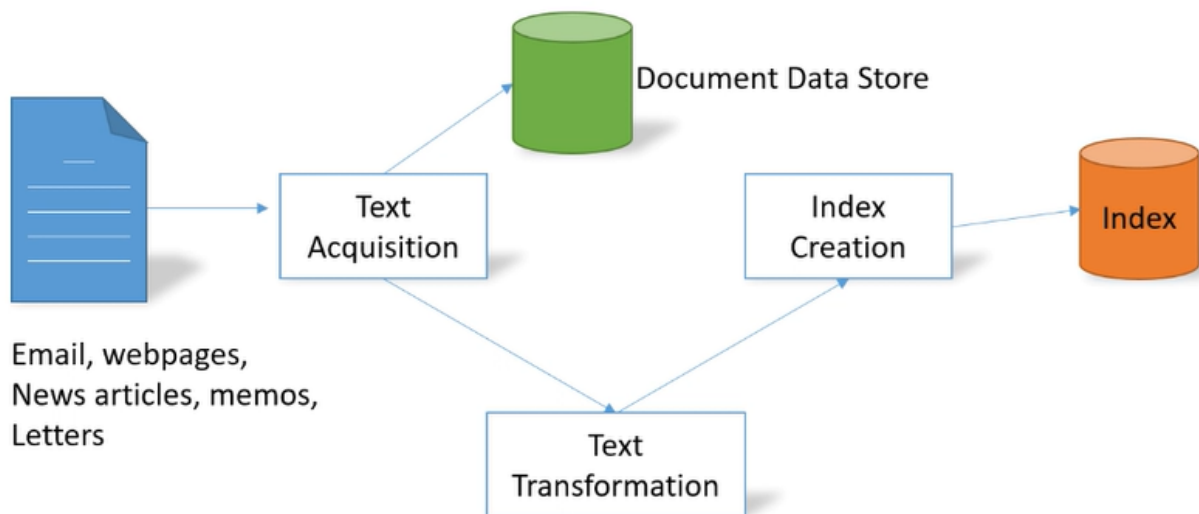
There are 2 main goals of a search engine:

- **Effectiveness** - only the highest quality, relevant data should be outputted
- **Efficiency** - Search engines should have a high query performance to output results quickly

Indexing Process:

(Don't worry too much about this)

The Indexing Process



Text Acquisition = Obtaining text content to be used as an index from a variety of web pages that users will search for and adding them to a database which users can query for efficient searching. This is performed by a crawler which extracts the text content from web pages and follows links to other web pages (to determine the most relevant web pages).

Feeds can also be used here where a website's feed (real time stream of additions to a web page) is monitored and any additions to it are added to the index, this allows for efficient indexing as crawlers will not have to traverse the entire web page

Conversion may also be performed where search engines convert acquired content into a consistent format to ensure consistency in the DB

Document Data Store = A DB used to store the documents (things found on the internet) being indexed and relevant structured data (metadata) about them locally, usually in a compressed form due to the large numbers of documents being indexed. A document database is most efficient for this as they provide efficient queries for large volumes of data, this is done as downloading all the documents each time they are crawled would use a lot of system resources

Text Transformation = Raw data extracted from web pages by text acquisition is prepared for efficient storage. This is done by using a parser to parse the text and convert it into a series of labels. Common words which are unlikely to be searched for on their own (e.g. 'or', 'the') will then be removed from the stream of labels to reduce the size of the index. Stemming will then be performed to group common words together by their stem (so 'runner' and 'running' will both be reduced to 'run') to account for plurals and tenses in searches.

Index Creation = Converting the stream of labels into a number of indexes for the web page they originated from, key words or groups of keywords are then identified and converted into indexes for the web page. They are then stored and compared with searches the user makes to return relevant web pages quickly to a user

The use of indexing ensures that a search engine is efficient, as indexes can be used to quickly query the Document Data Store for relevant web pages, and ensures that the search engine is effective as processes can be

performed (e.g. weighting) which determine the most relevant web pages based on a search

User Input:

A user must provide a search which will be parsed into a simple query by reducing a search to a series of keywords and boolean operators. Query expansion may also be used to suggest additional search terms

A search engine also must output a display of the ranked documents

Ranking:

A page ranking algorithm is used to rank web pages based on their relevance, this is done by scoring the web pages based on their relevance, with a higher scoring page being more relevant and being outputted first

Ranking algorithms must be efficient so that a search engine is efficient and outputs relevant results quickly

Evaluation:

Search engines must determine how well they are performing so they can be improved later and cope with changing data

They do this by logging the precision and recall values of searches to analyse how relevant the outputs are to a user

They also log the speed of queries to determine whether a query would be performed faster after a change

Web Page Retrieval:

A DNS server retrieves the URL entered by the user (or followed by crawler), the DNS server then translates the hostname into a corresponding IP address. The search engine then connects to a server with that IP address. Once a connection is made, the search engine (or crawler) requests the page in the URL from the server

Web Crawler:

Web Crawler = a program which traverses web pages, downloads the content of the web page to be used as indexes for that webpage and then follows links on the web page to other webpages to determine which web page is the most relevant

This can cause problems where less powerful servers use all their resources handling the requests of a crawler, instead of the requests of the users on the web page, this is handled by implementing a politeness policy where the crawler waits for some time before making subsequent requests to allow the server time to respond to the requests of users

Crawlers can be directed using the robots.txt file which defines where crawlers are allowed to crawl

Focused Crawling = a method of crawling where a number of websites are crawled, starting at a particularly popular page, to find web pages of a particular topic and only relevant web pages are crawled, as opposed to crawling all web pages indiscriminately

This leads to more efficient crawling when concerned about a particular topic

Sitemaps can also be used to define how often a web page is updated so crawlers do not waste time crawling a web page which has not been updated, making the crawling process more efficient

Deep Web:

Deep Web = sections of the internet which crawlers struggle to access

These web pages are difficult to crawl because:

- An account is needed to view the web page, crawlers are programs so cannot create an account
- Web pages which show after an action is completed, e.g. pressing button to visit next page, crawlers cannot complete these actions
- Web pages which use excessive javascript, crawlers cannot interpret javascript

Document Feeds:

Document Feed = if a document is created then not updated at all, they can be ordered in order of their creation so web crawlers can easily find new documents by simply identifying the new documents at the end of the feed

